개발완료보고서 · 2026

IcePredict

AI 예측과 RTOS 반응의
이중 안전망 블랙아이스 대응 시스템

AI가 노면을 미리 보고, 못 보더라도 RTOS가 차량 거동으로 확실히 잡는다.

제24회 임베디드SW경진대회 · 자동차/모빌리티 부문 (현대자동차)

팀 ARTS · 이지성 · 남윤상 · 김진찬

STM32N6570-DK · Neural-ART NPU · ThreadX · Raspberry Pi 5 + AI HAT+ 2 · CARLA HIL

github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS



Moza 휠·페달로 사람이 CARLA 를 직접 운전하는 HIL 시연. 운전자가 바뀌어도 보드는 같은 판정을 한다.

한 장 요약

AI가 노면을 미리 보고, 못 보더라도 RTOS가 차량 거동으로 확실히 잡는다.

문제

블랙아이스는 공개 데이터에 라벨이 없다. 투명한 얼음은 아스팔트와 시각적으로 구분되지 않는다. 카메라 한 겹으로는 반드시 놓치는 경우가 남는다.

해법

예측과 반응을 한 보드에 두 겹으로 올렸다. 1차는 NPU 비전이 빙판에 닿기 전에 세우고, 1차가 놓치면 2차가 IMU 거동으로 미끄러짐을 확정해 받는다.

증명

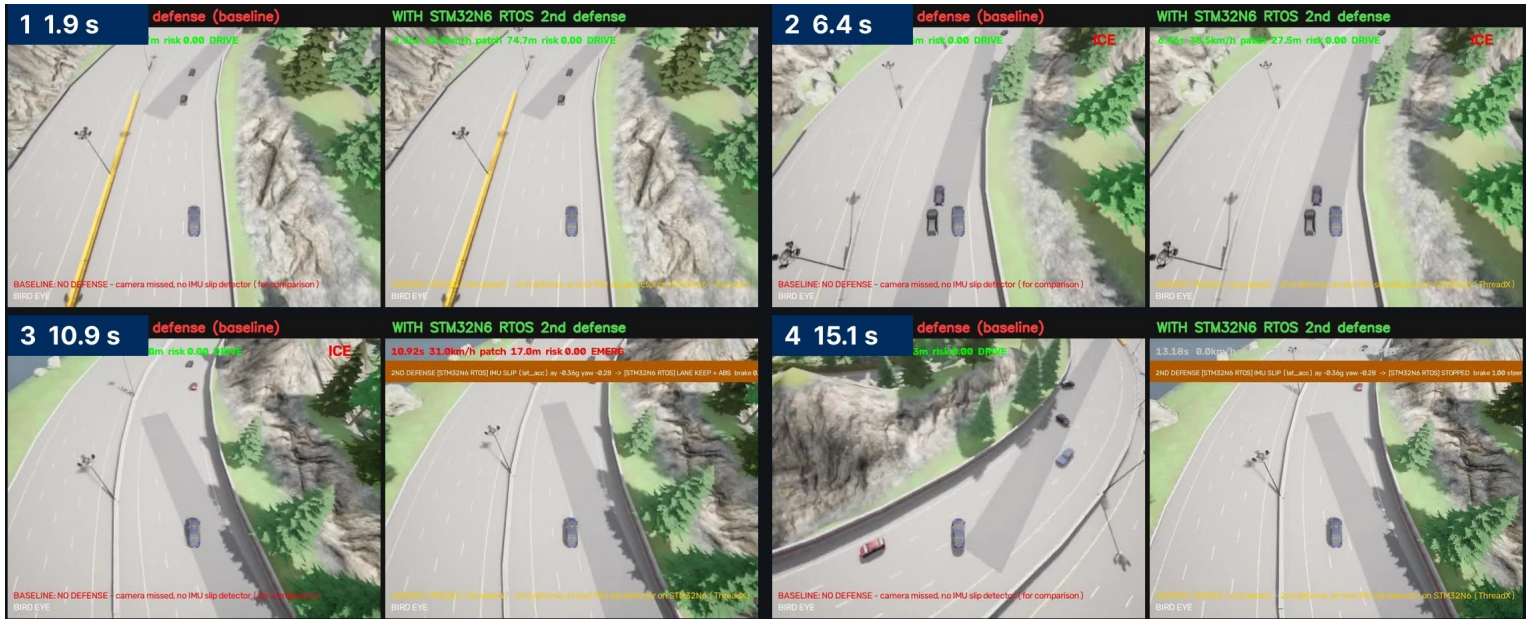
실물 STM32N6 보드가 실제 펌웨어를 돌리고 CARLA 가 센서·물리를 제공하는 HIL 로 주행 108회, 실사진 69,358장을 보드에 직접 넣어 측정했다.

96.1%
실사진 블랙아이스 6,340장 정답률
경보율 96.6 %

0.3%
마른 노면 19,018장 오경보율
젖은 노면 0.8 %

22.6 µs
2차 방어 최악 응답
리눅스는 5,790 µs

9/9
방어를 모두 끈 기준선
전부 제어 상실



주요 장면 캡처 4컷 (영상은 저장소에서 재생) · 조감 비교 16 s

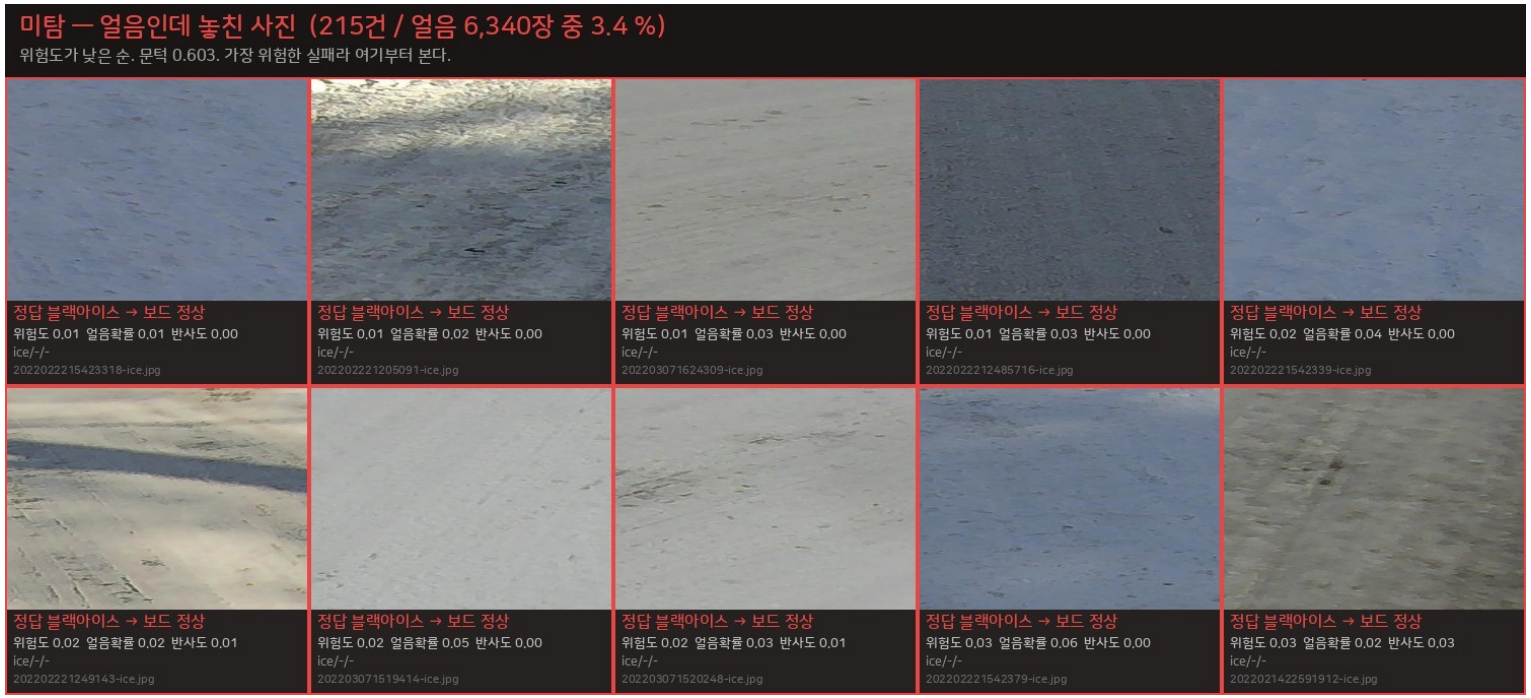
같은 빙판 · 같은 속도
같은 주변 차량 8대

25.4 km/h
왼쪽 · 방어 없음
제어를 잃고 차로를 가로질러 미끄러진다

0.0 km/h
오른쪽 · STM32N6 RTOS 2차 방어
차선 안에서 정지했다

실물 STM32N6 보드가 ThreadX 위에서 IMU 거동으로 미끄러짐을 확정하고 제동·조향을 내린다. 차이는 2차 방어 하나뿐이다.

왜 블랙아이스는 비전만으로 풀리지 않는가



우리 보드가 실제로 놓친 실사진 — 전부 정답이 블랙아이스인데 '정상'으로 판정했다. 위험도 0.01~0.04 라 운영 문턱(0.603)은 물론 어떤 문턱으로도 잡히지 않는다

3.4 %
실사진 얼음 6,340장 중 215장이
어떤 문턱으로도 잡히지 않는다

0 장
공개 데이터셋의 '블랙아이스' 라벨
RSCD ice 57,262장도 다져진 눈·서리에 가깝다

2 겹
그래서 예측(AI)과 반응(RTOS)을
한 보드에 두 겹으로 올렸다

라벨이 없다

공개 데이터셋에 '블랙아이스' 라벨이 없다. RSCD 의 ice 57,262장도 다져진 눈·서리에 가까운 '얼음 노면'일 뿐, 투명한 블랙아이스를 따로 구분하지 않는다.

원래 어려운 문제다

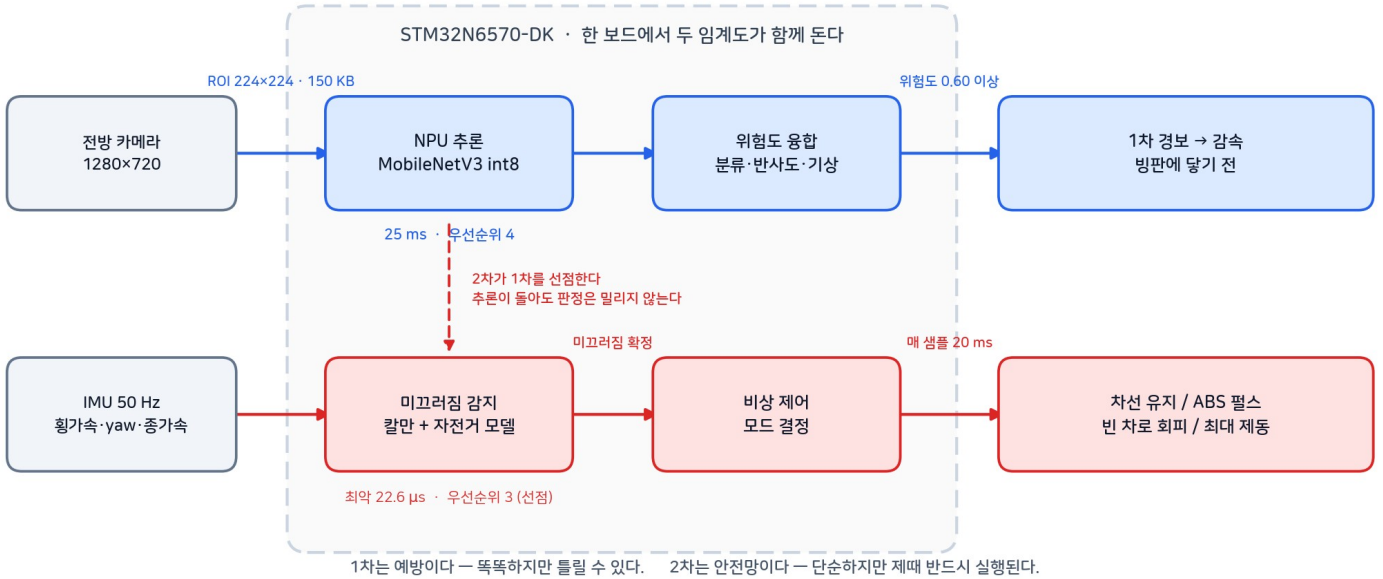
투명·검은 얼음이 아스팔트와 시각적으로 구분되지 않는다는 것은 업계 통설이다. 우리 모델도 마찬가지로, 실사진 얼음 6,340장 중 215장(3.4 %)이 문턱 아래에 남는다.

그래서 내린 결론

1차는 블랙아이스를 직접 맞히려 하지 않는다. '결빙 위험 노면 확률 + 반사도 이상 + 기상 맥락'으로 위험도를 올린다. 그리고 그 전략이 실패할 때를 위해 2차가 있다.

약점을 숨기지 않고, 이중 안전망 구조의 '존재 이유'를 데이터로 뒷받침한다 — 개발계획서에 적었던 방침을 그대로 지켰다.

이중 안전망 — 예측(AI)과 반응(RTOS)



25 ms

1차 방어 · STM32N6 Neural-ART NPU
노면 4분류 + 반사도 추론 한 프레임

22.6 µs

2차 방어 · ThreadX 최악 응답
IMU 샘플 → 미끄러짐 판정 → 제어 출력

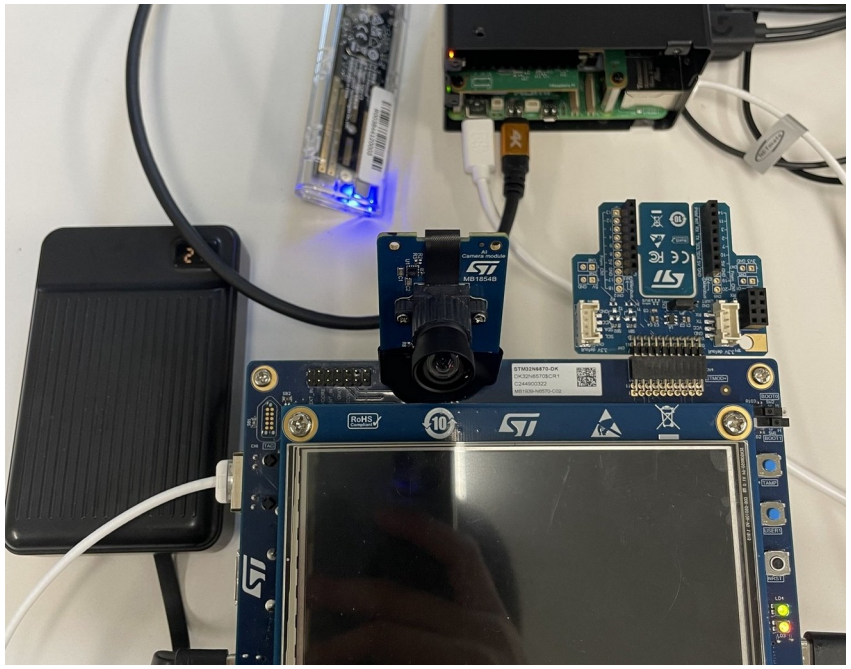
3 › 4

ThreadX 우선순위 — IMU 융합 3, NPU 프레임 4
추론이 돌고 있어도 IMU 판정이 선점한다

	1차 방어 · 예측	2차 방어 · 반응
센서	전방 카메라	IMU (횡가속 · yaw rate · 종가속)
판단	노면 4분류 + 반사도 + 기상 맥락 → 위험도	칼만 필터 + 자전거 모델 잔차 → 미끄러짐 확정
시점	빙판에 닿기 전	미끄러지기 시작한 뒤
성격	똑똑하지만 틀릴 수 있다	단순하지만 제때 반드시 실행된다
실행	STM32N6 NPU · 25 ms	STM32N6 + ThreadX · 최악 22.6 µs

ThreadX 우선순위는 IMU 융합 스레드가 3, NPU 프레임 스레드가 4다. 25 ms짜리 추론이 돌고 있어도 IMU 판정이 선점한다 — 혼합 임계도(mixed-criticality) AI ECU 구조.

하드웨어 구성



STM32N6570-DK + AI 카메라 MB1854B — 왼쪽은 보조배터리, 위는 NVMe SSD



Raspberry Pi 5 16 GB + AI HAT+ 2
KKSB 케이스 · 액티브 쿨러



RealSense D435i (RGB·뎁스·IMU)
NVMe 1 TB SSD 외장

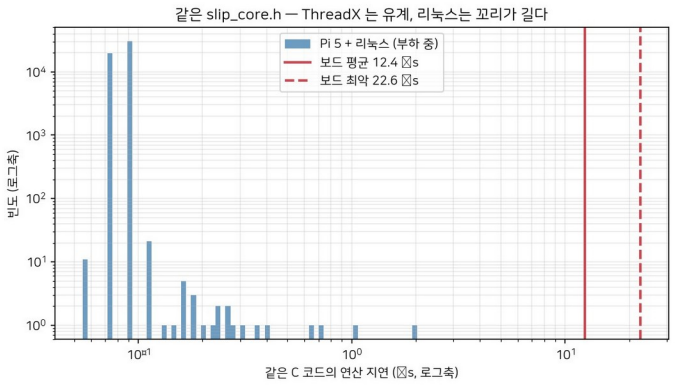


Moza 휠 · 페달 — 사람이 직접
CARLA 를 운전하는 경로

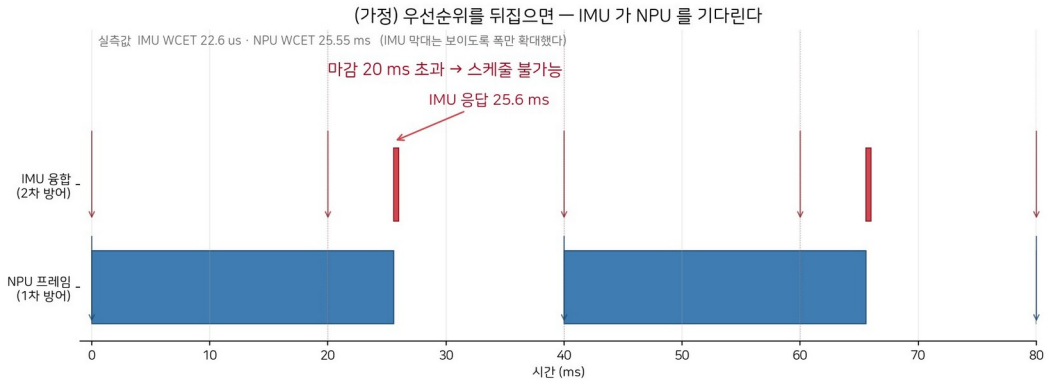
구분	정식 명칭	역할
메인 보드	STM32N6570-DK (MB1939-N6570-C02 + MB1860B)	1차 NPU 추론(Neural-ART @ 1 GHz) + 2차 ThreadX 실시간 판정·제어. 두 임계도가 한 보드에 공존한다
카메라	ST AI Camera module MB1854B	1차 방어 입력. 보드에 직결되어 NPU 로 바로 들어간다
호스트	Raspberry Pi 5 16 GB + AI HAT+ 2 (Hailo-10H)	기상 맥락 생성, ZMQ 브리지, 교차 검증용 NPU (대회 필수 보드)
저장·전원	NVMe 1 TB SSD 외장 · 보조배터리	데이터셋·주행 로그 적재 / 콘센트 없이 같은 구성을 돌리기 위한 것
센서·렌더	Intel RealSense D435i · RTX 5090	실측 IMU 노이즈 확보 / CARLA 0.9.16 물리·센서 제공

개발계획서 대비 변경점과 그 이유

항목	개발계획서(6월)	실제 구현	바꾼 이유
2차 방어 실행 환경	Pi 5 + 리눅스	**STM32N6 + ThreadX RTOS**	리눅스 최악 깨어남 지연이 **5,790 µs**로 측정됐다. 제어 주기 20 ms의 29 %를 한 번의 지터가 먹는다. 평균이 아니라 꼬리가 안전 기능의 기준이다
1차 NPU	Hailo-10H 단독	**Neural-ART(N6) 주** · Hailo 교차검증	Hailo Dataflow Compiler 를 확보하지 못해 int8 변환 경로를 열지 못했다. 1·2차를 한 보드에 올리는 편이 선점 구조를 증명하기에도 맞다
검증 방법	1/5 차량 저마찰 노면 실측	**HIL** (실물 보드 + CARLA) + 실사진 69,358장	실물 빙판을 재현 가능하게 만들기 어렵다. 대신 펌웨어는 실물 보드에서 그대로 돌리고, 비전 성능은 시뮬 화면이 아닌 실제 도로 사진으로 측정했다
노면 클래스	정상 / 젖음 / 결빙 위험 / 포트홀	동일 (유지)	RSCD 27클래스 → 4클래스 매핑을 계획대로 적용



1번 변경의 근거 — 같은 연산의 지연 분포. 리눅스는 꼬리가 길다



2번 변경의 근거 — 우선순위를 뒤집으면 NPU 추론(25 ms)이 IMU 주기(20 ms)를 막는다

바뀌지 않은 것

이중 안전망이라는 구조와, '블랙아이스를 직접 맞히지 않고 위험도를 올린다'는 1차 전략, '노면 라벨과 무관하게 차량 거동으로 확정한다'는 2차 전략은 계획서 그대로다. 바뀐 것은 전부 그 구조를 더 확실히 증명하기 위한 수단이다.

데이터 전략 — 모델 헤드별 정답을 먼저 정의했다

데이터셋을 먼저 고르지 않았다. 각 헤드가 무엇을 정답으로 배워야 하는지를 정의한 뒤 거기에 맞는 공개 데이터셋을 매핑했다.

방어	필요한 정답(label)	데이터셋	활용
1차	건조 / 젖음 / 눈 / 결빙 클래스	**RSCD**	주 학습 (사전학습)
1차	실측 수막 두께 (센서 GT)	**RoadSaW**	반사도 헤드 근거
1차	눈 · 비 · 안개 + 조도	**AI Hub**	국내 도로 도메인 검증
2차	실측 6축 IMU + 노면 라벨	**PVS**	칼만 공분산 설정
2차	RGB-IMU 동기화 · 악조건	**ROAD**	비교 기준선
보조	실제 폭설 주행 씬	**CADC · WADS**	야간 · 악천후 점검
보조	자동 라벨 (차선 가시성)	**CARLA**	합성 데이터 생성

RSCD 27클래스 → 우리 4클래스

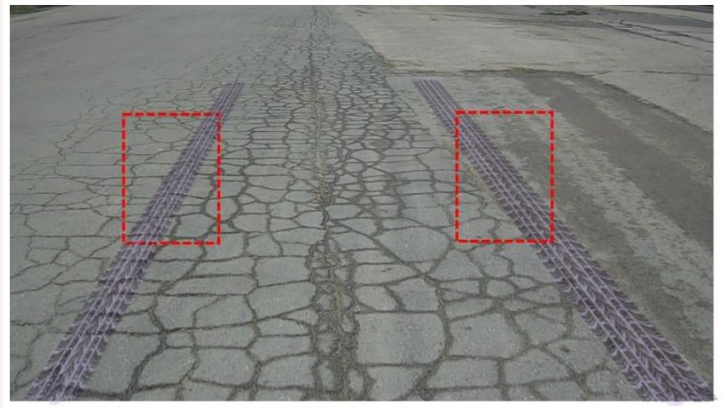
정상 ← 건조 (dry)　　**젖음** ← 젖음 · 물 고임 (wet · water)

결빙 위험 ← 결빙 · 녹은 눈 (ice · melted snow)　　**포트홀** ← 요철 '심함' 라벨

약 100만 장(공개 서브셋 37만 장) · 27클래스 = 마찰 6 × 재질 4 × 요철 3 · 베이징 약 700 km 실도로 주행(2022) · 240×360 패치라 NPU 입력 크기에 맞는다 · 눈·얼음은 결빙 57,262 · 녹은 눈 64,263 · 신설 76,730장



RSCD 클래스별 샘플



주행 영상에서 노면 영역만 잘라 패치로 쓴다

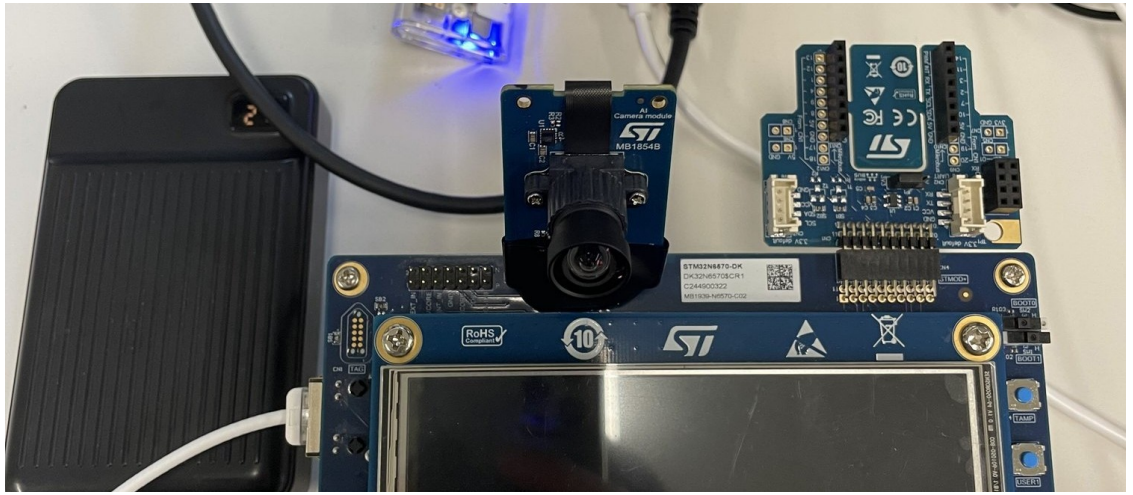


차량 전방 카메라 — 20~80 km/h 주행 촬영

데이터셋 그림은 각 공개 데이터셋의 논문·공식 사이트에서 가져온 설명 목적의 인용이며, 라이선스는 저장소 NOTICE 에 명시했다.

모델과 int8 배포 — RoadNet

항목	값
백본	MobileNetV3-Small (ImageNet 사전학습)
입력	1×3×224×224 NCHW, int8
출력	4클래스 로짓 + 반사도 헤드 (576→64→1)
내보내기	ONNX opset 13, 고정 배치 1
양자화	int8 PTQ (QDQ), 가중치 채널별 · 활성 대칭
입력 양자화	scale 0.018658448, zero-point −14
출력 양자화	scale 0.029451849
프레임당 전송	150,528 B
NPU	Neural-ART @ 1 GHz · 추론 **25 ms**
카메라 ROI	차량 전방 **7.9 ~ 42.2 m** (FOV 60°, 피치 −12°)



이 모델이 실제로 돌아가는 보드 — STM32N6570-DK 위의 AI 카메라 MB1854B 가 NPU 입력을 바로 공급한다

반사도 헤드를 따로 둔 이유

블랙아이스의 단서는 '무슨 노면인가'보다 '빛을 어떻게 되돌리는가'에 가깝다. 분류 로짓 하나로는 젖음과 결빙이 섞인다. RoadSaW 의 실측 수막 두께를 정답으로 반사도를 회귀로 따로 배우게 하고, 융합 단계에서 분류 확률과 더한다.

양자화에서 겪은 것

표준 PTQ 로는 stem 층의 활성 분포가 넓어 int8 에서 정확도가 떨어졌다. stem 등화(equalization)와 클리핑을 적용해 회복했고, QDQ 그래프를 u8→i8 로 다시 쓰는 변환을 거쳐 Neural-ART 가 받는 형태로 맞췄다.

배포 경로

01
학습
PyTorch · RSCD 패치
MobileNetV3-Small

02
내보내기
ONNX opset 13
고정 배치 1

03
양자화
int8 PTQ (QDQ)
stem 등화 + 클리핑

04
보드
u8→i8 변환
Neural-ART @ 1 GHz

성능 — 실제 도로 사진 69,358장을 보드에 직접 넣었다

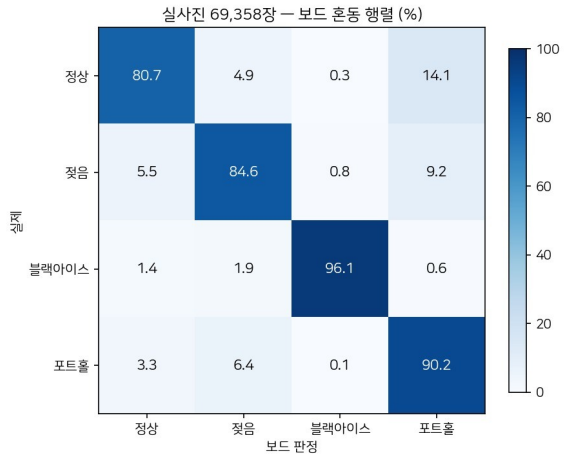
시뮬레이션 화면이 아니다. RSCD 실사진을 STM32N6 에 그대로 넣고 추론·융합을 전부 보드에서 돌려 받은 판정이다.

실제 노면	장수	정답률	경보율 (문턱 0.603)
블랙아이스	6,340	**96.1 %**	**96.6 %**
마른 노면	19,018	80.7 %	**0.3 %**
젖은 노면	34,440	84.6 %	**0.8 %**
포트홀	9,560	90.2 %	**0.3 %**

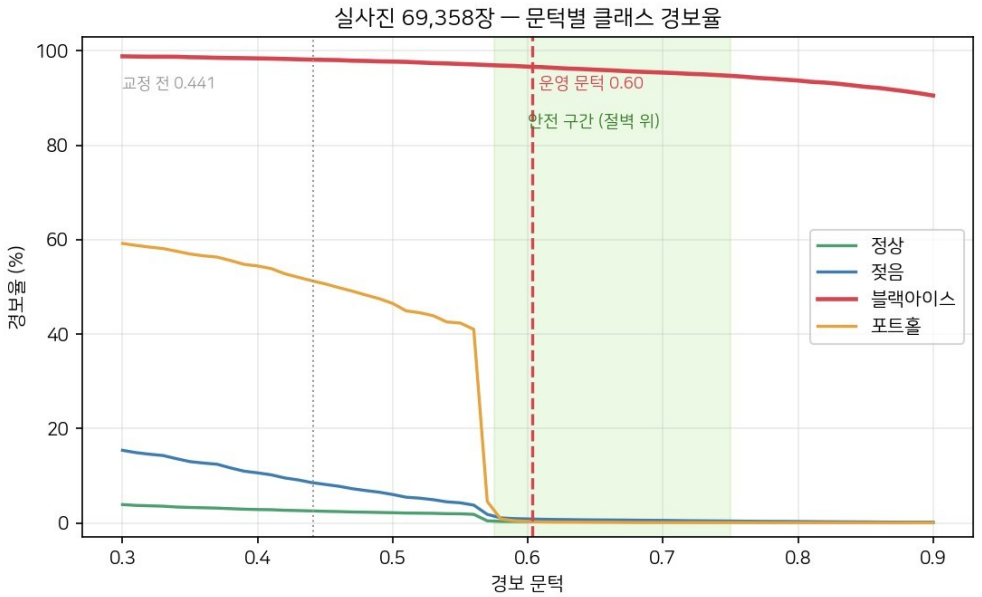
96.6 %
블랙아이스 6,340장
경보율

0.3 %
마른 노면 19,018장
오경보율

0.8 %
젖은 노면 34,440장
오경보율



혼동 행렬 (행 = 실제, 열 = 보드 판정)



운영 문턱 결정 — 69,358장 전량 스윕. 절벽 구간을 찾아 0.603 으로 정했다

이 숫자가 왜 믿을 만한가

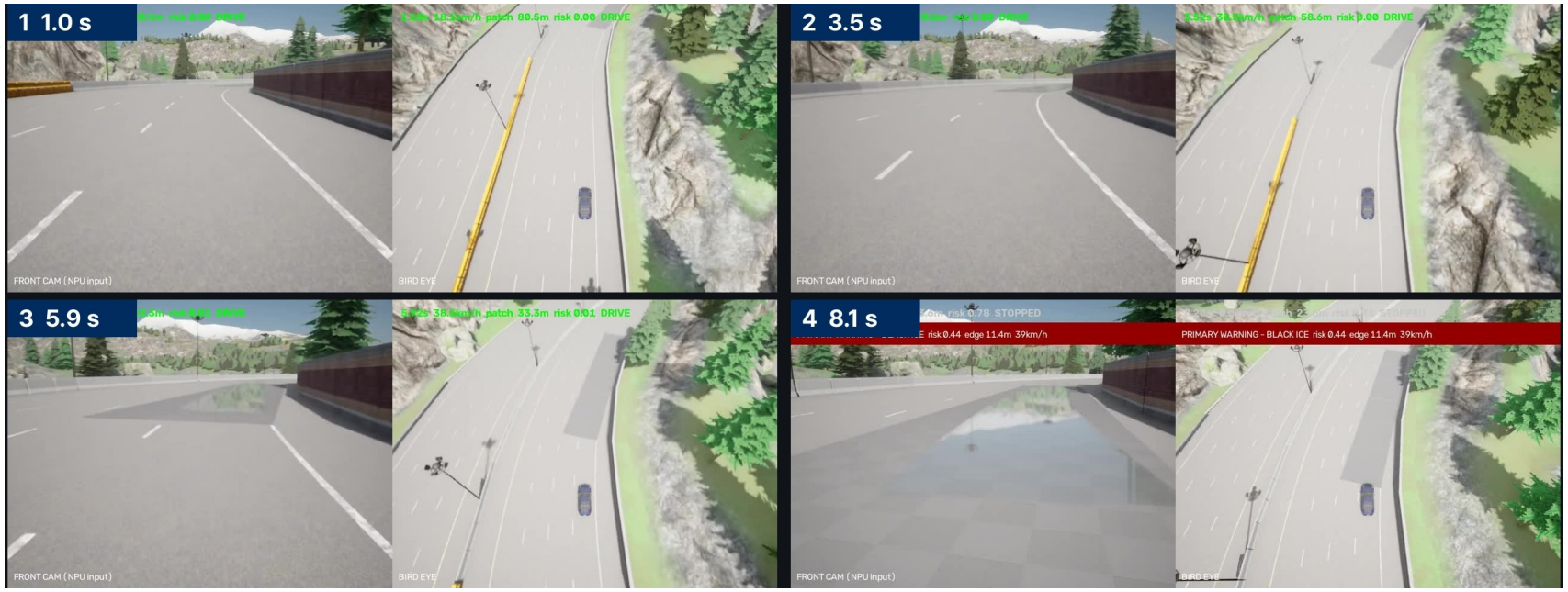
표본이 보드 바깥에서 만들어진 것이 아니다. 실사진을 STM32N6 에 그대로 넣어 추론·융합을 보드에서 돌리고, 돌아온 판정을 그대로 셌다.

장당 판정을 `logs/rscd_board_samples.jsonl` 에 남겨 두어, 문턱을 바꿔 다시 계산할 때 보드를 다시 돌릴 필요가 없다.

경보율은 보드의 alarm 플래그가 아니라 risk ≥ 문턱으로 계산했다. 보드 융합기의 히스테리시스(켜짐 0.441 / 꺼짐 0.341)는 연속 영상의 깜빡임을 막지만, 서로 무관한 낱장 사진을 이어 넣으면 앞 사진 상태가 넘어와 경보율이 부풀려지기 때문이다.

동작 — 카메라가 보고 빙판 앞에서 선다

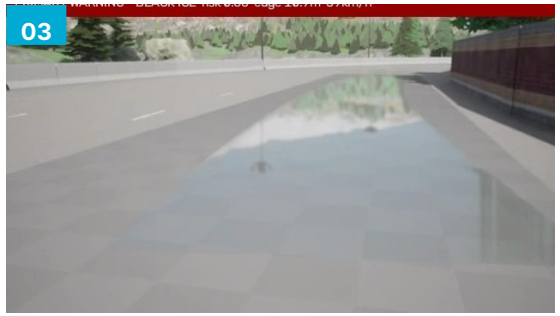
CARLA Town04 · 맑은 낮 · 40 km/h · 마찰 0.02 · 1차 방어 ON (보드 NPU 추론 + 융합)



주요 장면 캡처 4컷 (영상은 저장소에서 재생) · 왼쪽 전방 카메라(NPU 입력) · 오른쪽 조감







01 접근

40 km/h 주행. 왼쪽은 전방 카메라(모델 입력), 오른쪽은 조감.
위험도 0.00 · **DRIVE**

02 1차 경보 6.10 s

위험도 0.443 → 문턱 돌파. 빙판 가장자리까지 11.4 m.
PRIMARY WARNING · BRAKE

03 정지 7.14 s

빙판 23.6 m 앞에서 멈췄다.
빙판에 닿지 않았다 — **예방에 성공**

같은 주행선에서 날씨 11종(맑음·흐림·젖은 노면·보슬비·폭우·해질녘·밤·비 오는 밤·눈 등)을 모두 돌렸다. 인식·정지 14회 · 경고 늦음 7회 · 미인식 10회 · 오경보 1회.

위험도 융합 — 기상·위치 맥락으로 가중치를 바꾼다

$$\text{risk} = \alpha \cdot p_ice + \beta \cdot \text{반사도} + \gamma \cdot (1 - \text{차선 가시성})$$

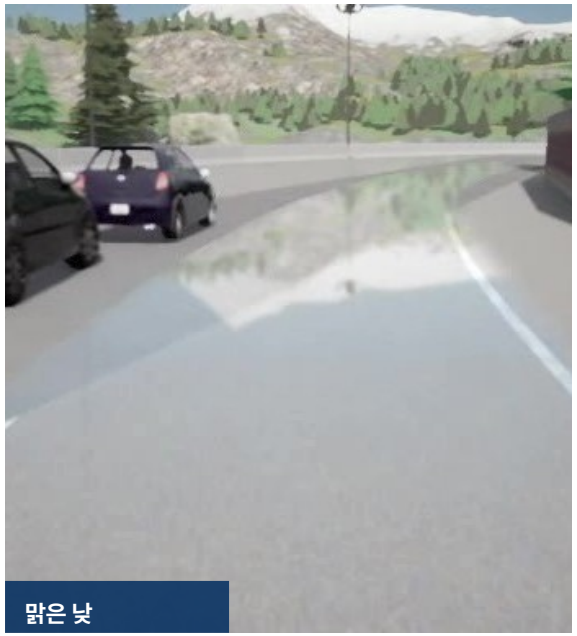
기상 · 위치 맥락	α (분류)	β (반사도)	γ (차선)
교량 · 새벽 등 결빙 위험 높음	0.35	**0.45**	0.20
일반	**0.50**	0.30	0.20
저위험	**0.55**	0.15	0.30

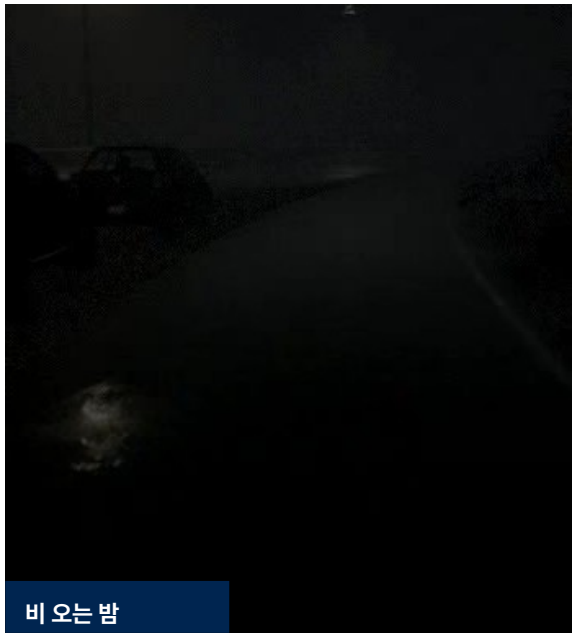
설계에서 고친 두 가지

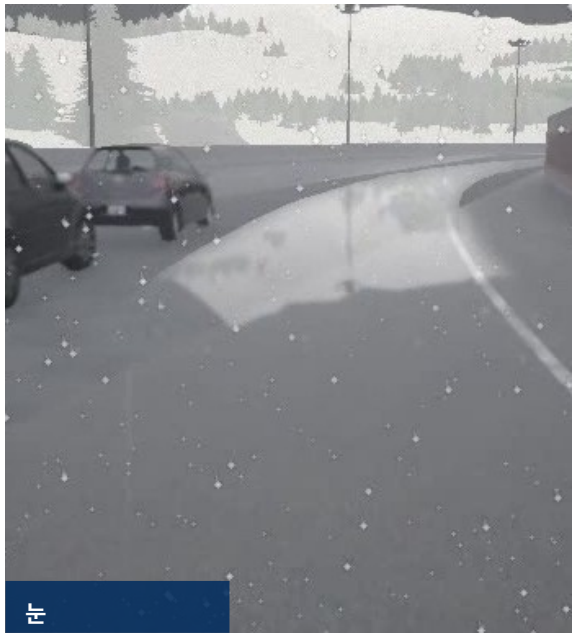
① **재정규화.** 쓸 수 없는 신호에 0을 넣으면 안 된다. 0은 중립값이 아니라 최솟값이라 위험도 상한이 잘린다. 그래서 쓸 수 없는 신호는 분모에서 빼고 남은 신호로 다시 정규화한다.

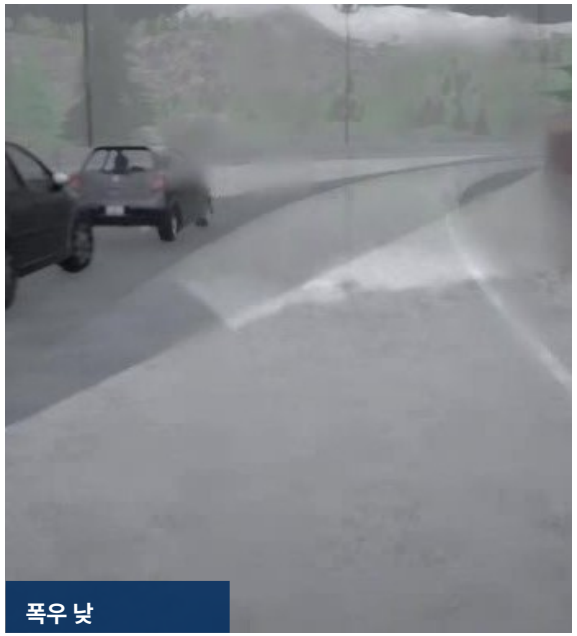
② **문턱 재교정.** 초기 0.441 에서는 젖은 노면 오경보가 40 %였다. 실사진 69,358장 스윕으로 절벽 구간을 찾아 0.603 으로 올렸고 오경보가 0.8 %로 내려갔다. 폭우는 강수 게이트로 따로 분리했다.

맥락 계층이 보는 입력 — 같은 빙판 · 같은 주행선, 날씨만 바꿨다









운영 문턱 0.603 — 손으로 고른 값이 아니라 실사진 전량 스윕에서 결정했고, logs/rscd_board_samples.jsonl 로 문턱을 바꿔 재계산할 수 있다.

원리 — 칼만 필터 + 자전거 모델 잔차

판단 절차

1 IMU 50 Hz 에서 횡가속 a_y · yaw rate · 종가속을 받는다

2 2상태 칼만 필터(값 · 변화율)로 잡음을 거른다 — Q = 2000/3000, R = 0.16/0.0005

3 자전거 모델로 '이 속도·이 조향이면 나와야 할 yaw rate'를 계산한다 (고속 언더스티어 보정 1/(1+(v/v_ch)²), v_ch = 17 m/s)

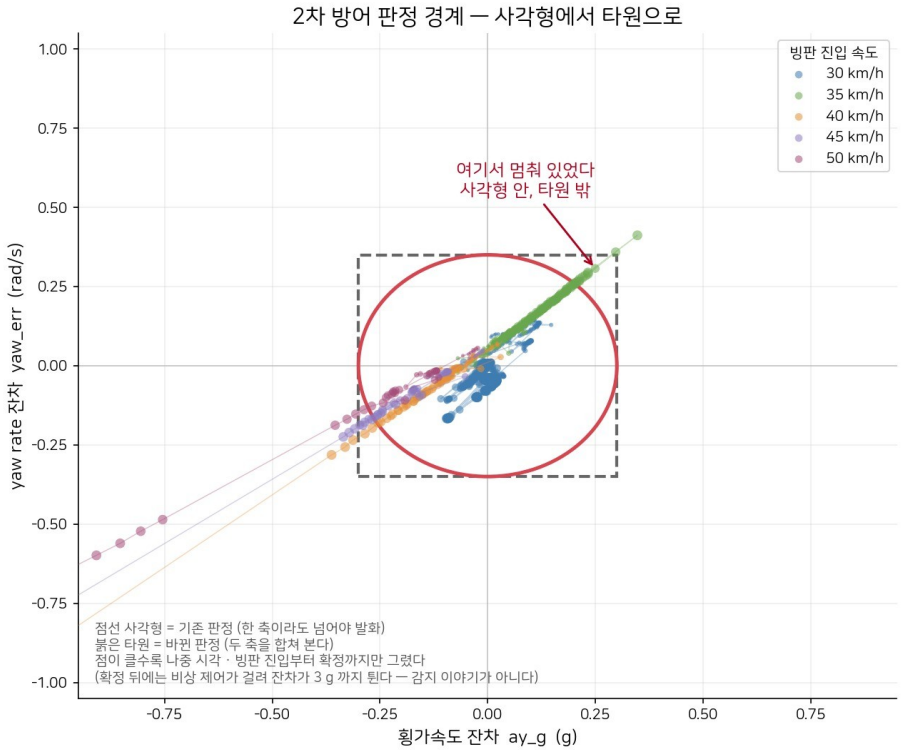
4 실측과 모델의 잔차가 **타원 밖**이면 미끄러짐 — 3샘플(60 ms) 연속이면 확정

항목	값	왜 이 값인가
축거 L	2.7 m	자전거 모델
조향 지연 τ	**0.06 s**	급조향 시 '모델이 즉답한다'는 가정이 오탐을 만들었다
횡가속 임계	0.30 g	
yaw 오차 임계	0.35 rad/s	
판정 규칙	**타원**	직사각형은 모서리에서 저속 지연이 생겼다
저마찰 트리거	제동 ≥ 0.3 이 0.3 s 지속 + 감속 < 1.2 m/s²	필터 지연 구간의 오탐 방지

50 Hz
IMU 입력 주기

60 ms
3샘플 연속이어야 확정

22.6 µs
샘플 → 판정 → 제어, 최악



판정 경계 — 직사각형(점선) 대 타원(실선). 모서리에 걸리던 저속 구간이 타원에서 사라진다

C 코어(sw/fw/npu_lib/slip_core.h)는 HAL·OS 비의존이라 호스트에서도 컴파일된다. 파이썬 참조 구현과 같은 판정을 내리는지 매 커밋 검증한다.

왜 리눅스가 아니라 RTOS 인가 — 평균이 아니라 꼬리

같은 연산을 Pi 5 리눅스가 평균 146배 빠르게 한다. 그런데도 RTOS 보드를 쓴다. 안전 기능의 기준은 평균이 아니라 최악이기 때문이다.

플랫폼	표본	중앙값	최악
Pi 5 + Linux · 유휴	20,000	69 µs	**5,790 µs**
Pi 5 + Linux · 부하	20,000	68 µs	**5,429 µs**
STM32N6 + ThreadX (응답 전체)	49,405	12.4 µs	**22.6 µs**

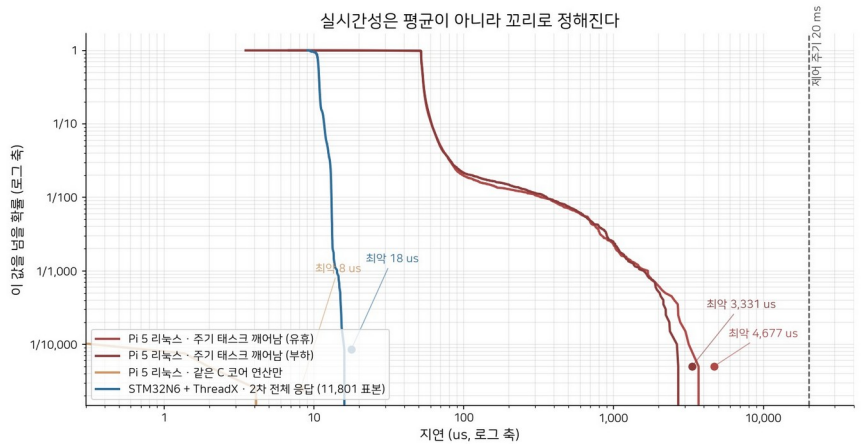
5,790 µs
Pi 5 리눅스
최악 깨어남 지연

22.6 µs
STM32N6 + ThreadX
최악 응답 전체

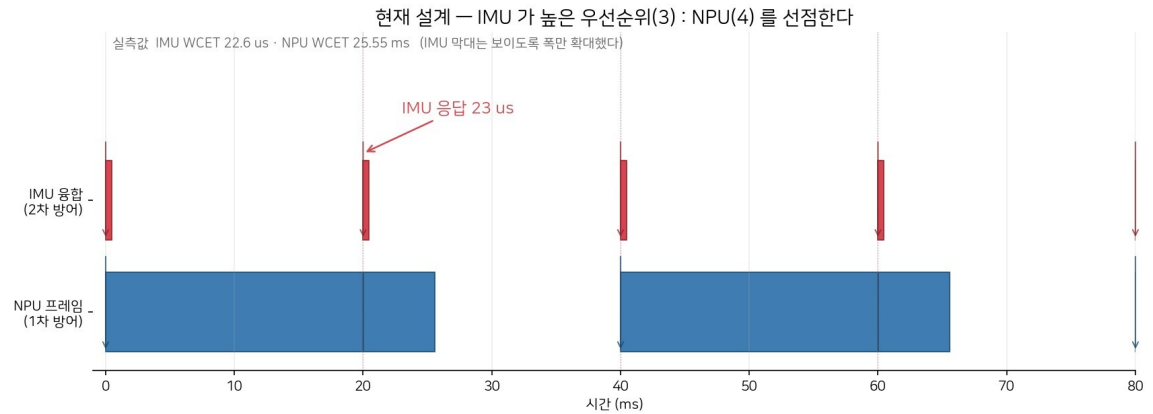
29 %
20 ms 제어 주기 중
지터 한 번이 먹는 몫

이것이 왜 치명적인가

제어 주기는 20 ms 다. 리눅스의 최악 지터 5,790 µs 는 한 주기의 29 %를 한 번에 먹는다. 40 km/h 에서 5.8 ms 는 6.4 cm 지만, 미끄러짐이 시작된 뒤의 제어 루프에서는 그 한 번이 차선 유지와 제어 상실을 가른다.



지연 분포 CDF — 리눅스는 꼬리가 길다



RM 스케줄 — IMU(우선순위 3)가 NPU(4)를 선점한다

우선순위를 뒤집으면(NPU가 IMU보다 높으면) 25 ms 추론이 20 ms 주기를 막아 스케줄 자체가 불가능해진다 — 근거: sw/docs/evidence/10_스케줄가능성_분석.md

같은 조건 직접 비교 — 방어 없음 vs STM32N6 RTOS 2차 방어

같은 날씨 · 같은 주행선 · 같은 빙판 · 같은 주변 차량 8대. 차이는 2차 방어 하나뿐이다.



주요 장면 캡처 4컷 (영상은 저장소에서 재생) · 각각 전방 카메라(NPU 입력) + 조감 · 16 s

왼쪽 · 방어 없음

25.4 km/h

15.0 s · 12.0 s 에 차선을 이탈해 제어를 잃었다

차체가 진행 방향과 어긋난 채 차로를 가로질러 미끄러지는 중이다. 뒤따르던 차량이 그대로 접근하고 있다.

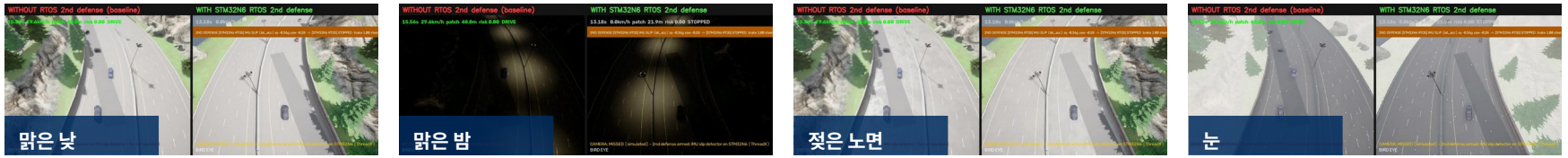
오른쪽 · STM32N6 RTOS 2차 방어

0.0 km/h

13.2 s · IMU SLIP (lat_acc) a_y −0.36 g, yaw −0.28 → STOPPED

차선 안에서, 빙판을 벗어난 지점에 정지했다. 미끄러짐 확정부터 정지까지 보드가 제동·조향을 냈다.

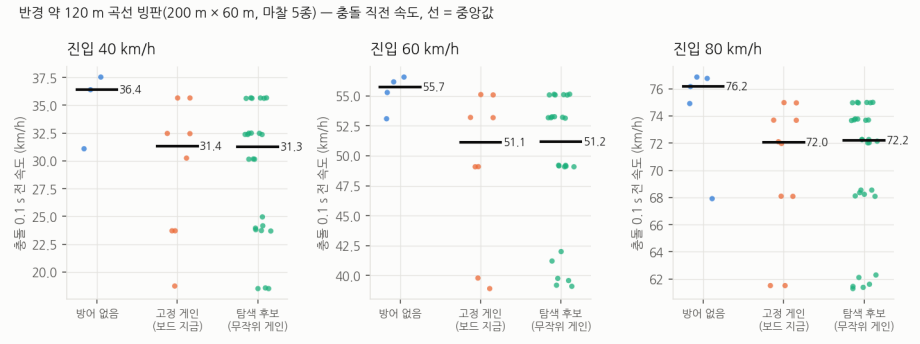
날씨 4종에서 같은 비교 — 각 쌍의 마지막 장면 (왼쪽 방어 없음 · 오른쪽 STM32N6 RTOS 2차 방어)



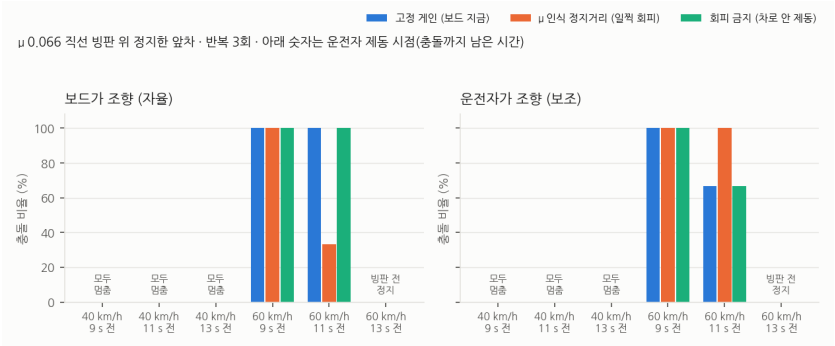
날씨 4종(맑은 낮 · 밤 · 젖은 노면 · 눈)에서 같은 비교를 만들었다. 전체 영상은 저장소 sw/media/ 와 logs/carla_demo/정리/G_비교_방어없음_vs_RTOS/ 에 있다.

2차 방어 확장 — '어떻게 밟나'가 아니라 '언제 밟나'

미끄러진 뒤의 제어를 더 다듬는 실험을 먼저 했다. 결과를 정한 것은 제어가 아니라 제동을 시작한 시점이었다.



① 곡선 빙판 — 마찰별 제어표 후보 78개가 고정 게인과 충돌 직전 속도 중앙값 0.2 km/h 안



② 직선 빙판 위 정지 차량 — 판단 방식을 바꿔도, 2차 방어를 꺼도 결과가 같다. 운전자 제동 시점이 정했다

찾은 것

· 빙판에서는 누가 밟든 감속이 마찰 한계(μg)에서 막힌다. 펄스 제동은 실차에서 ABS 가 이미 하는 일이다.

· 그래서 '브레이크를 어떻게 밟나'를 다듬어도 이길 여지가 없었다. 60 km/h 에서 9 s 전에 밟으면 전부 충돌, 13 s 전이면 섰다.

· 바꿀 수 있는 것은 **언제부터 밟기 시작하느냐**다.

	하는 일	못 하는 것
ABS · ESC	미끄러지는 바퀴를 잡고 차체 회전을 막는다	마찰을 다른 장치에 알리지 않는다
긴급제동 · 간격 유지	앞차 거리로 제동 시점을 정한다	마찰을 모르면 마른 노면 기준이라 빙판에서 늦다
노면 마찰 추정 (일부 양산차)	바퀴 신호로 미끄러움을 추정해 경고·공유	제동 시점까지 바로 잇는 경우는 드물다
IcePredict 2차 방어	**마찰을 직접 재서 제동 시작 시점을 노면에 맞춘다**	탭을 운전자가 느낀다 (다음 쪽)

근거: sw/docs/evidence/18_마찰별_게인스케줄링.md · 19_안전전용RTOS_연구계획.md. CARLA 0.9.16 시뮬레이션 결과다.

노면 확인 탭과 마찰 인식 제동 시점 (SLIP_ENVELOPE)

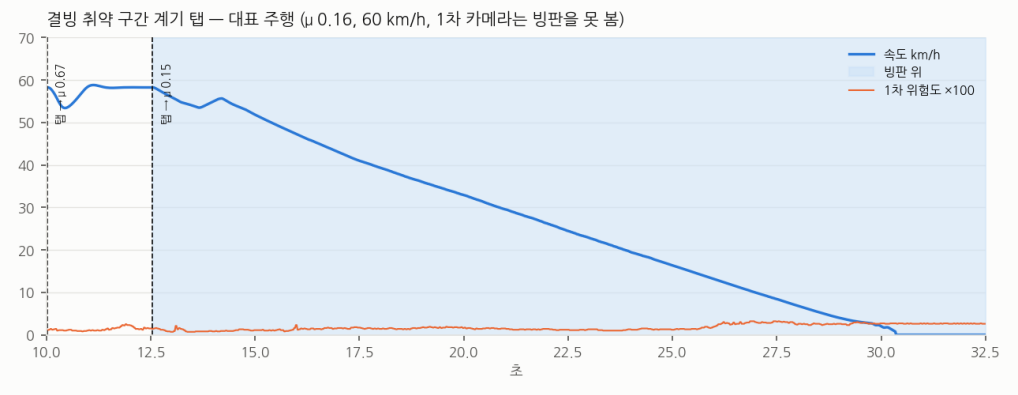
빙판은 살짝만 밟아도 감속이 거기서 막힌다. 그 막힌 값이 곧 노면 마찰이다. 무거운 계산은 CARLA 에서 미리 끝내고 보드는 표만 본다.

정지 차량 앞에서 제동을 시작해야 하는 거리 (m) — 2차 방어가 20 ms 마다 계산하는 값

	30 km/h	40 km/h	50 km/h	60 km/h	70 km/h	80 km/h
μ 0.03 매끈한 얼음	124	216	334	478	648	845
μ 0.07	67	114	176	250	338	439
μ 0.10	47	79	119	169	228	295
μ 0.16 블랙아이스	31	50	75	105	141	182
μ 0.25 다진 눈 정도	23	36	53	73	97	125
μ 0.41 젖은 노면 정도	16	25	35	48	63	80
μ 0.67 마른 노면	14	20	28	38	49	61

CARLA 0.9.16 실측 감속(제동 1.0) 기준 · 지연 0.3 s · 표 감속의 90 % · 여유 4 m. 비교 대상(마른 노면 기준)은 맨 아래 줄만 쓴다.

CARLA 실측 감속으로 계산한 '제동을 시작해야 하는 거리' — 60 km/h 마른 노면 38 m, 블랙아이스 105 m



대표 주행 — 결빙 취약 구간 첫 탭은 마른 노면 '이상 없음', 다음 탭이 빙판 μ 0.15

01 언제 확인하나

① 1차 카메라 위험도가 회색 구간(0.3 ~ 0.6)에 0.2 s → 카메라가 본 지점(19 m 앞)에 도착했을 때
② 영하 근처(3 °C 이하) 결빙 취약 구간 안 → 2.5 s 마다 최대 3번, 빙판을 찾으면 멈춤
안전 조건: 30 km/h 이상 · 직진 · 운전자 제동 없음 · 앞차 여유

02 노면 확인 탭

가속 페달을 0.2 s 떼고 브레이크 0.3 을 0.4 s. 마지막 0.25 s 평균 감속을 표로 되돌려 μ 를 잰다.
마른 노면이면 60 ms 만에 끝낸다 (속도 손실 13 → 5 km/h). 측정이 이상하면 한 번 다시 잰다

03 20 ms 마다 판단

정지 거리 = 0.3 s × v + v^2 / (2 × 0.9 × 실측 감속) + 4 m 를 앞 물체 거리와 비교해
순항 · 가속 금지 · 예비 제동 · 전력 제동 · 정지 유지 중 하나. 예비 제동은 남은 거리에 맞춰 서는 감속을 실측 감속으로 따라간다

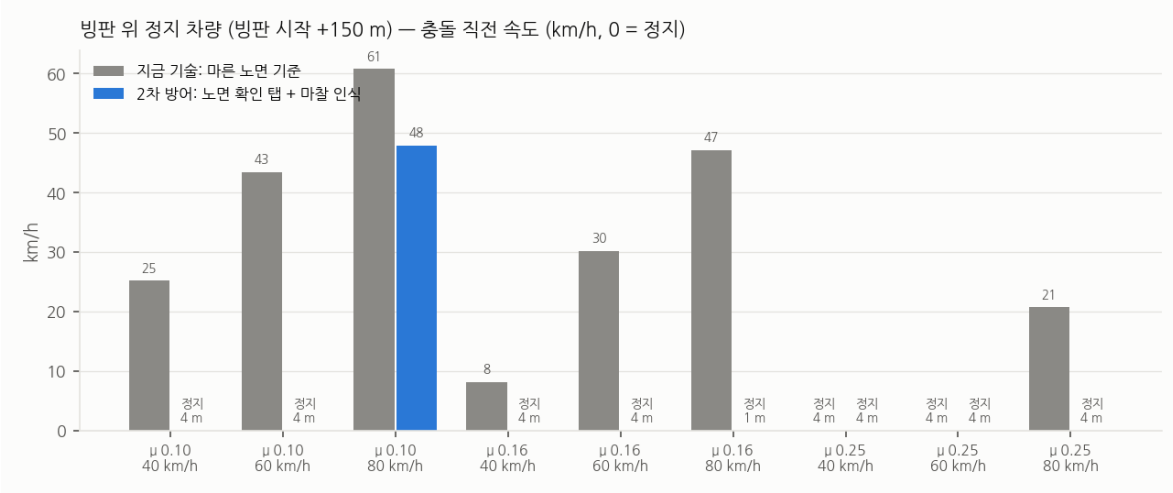
플래그 SLIP_ENVELOPE(기본 0)를 끄면 출력이 작업 전과 비트 단위로 같다. 단위 시험 11개 · C = 파이썬 = 파이 격리 코어 출력 일치. 근거: 20_마찰인식_제동시점.md

CARLA 비교 — 같은 C 코드, 마찰만 다르다 · 정지 2 / 9 → 8 / 9

간격 유지 크루즈(ACC) 주행 중 빙판 위 정지 차량. 비교 대상은 마른 노면 μ 를 가정하고, 2차 방어는 탭으로 잰 μ 를 쓴다.



■ 주요 장면 캡처 4컷 · 위: 마른 노면 기준 → 30 km/h 충돌 · 아래: 탭으로 마찰 측정 → 4.1 m 앞 정지



마찰 3종 × 속도 3종 — 충돌 직전 속도 (0 = 정지)

2 / 9
마른 노면 기준 정지
나머지 7회 8 ~ 61 km/h 충돌

8 / 9
마찰 인식 정지 — 7회 4.1 ~ 4.2 m 앞
(목표 여유 4 m)

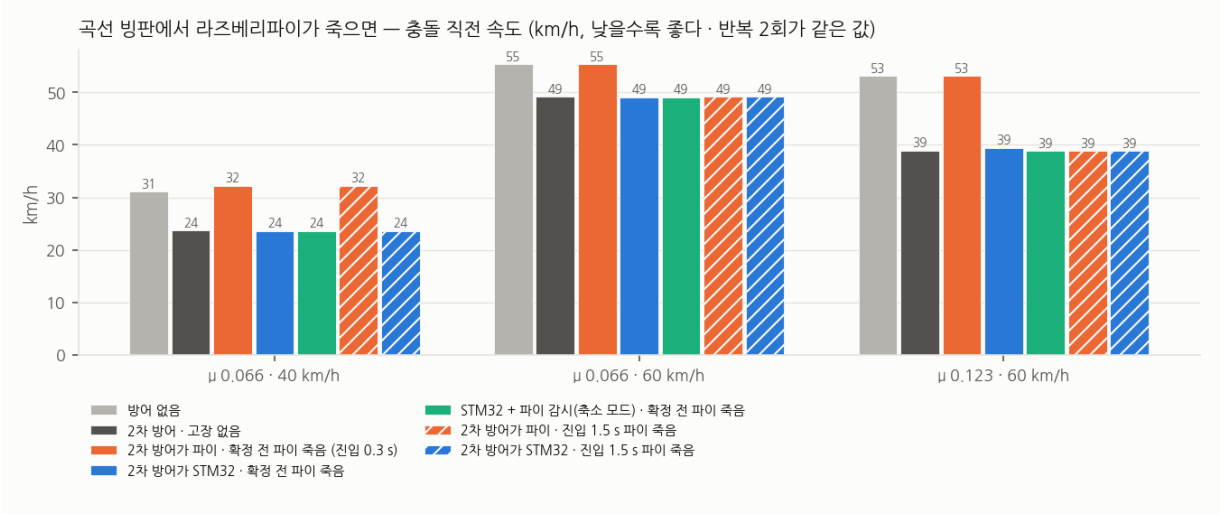
295 m
마찰 인식이 충돌한 1회 (μ 0.10 · 80 km/h)
필요 거리 > 150 m — 물리적으로 불가

탭 판단 시험 (1차 모델이 실제로 판정)	결과
빙판 + 결빙 취약 구간 · 맑은 낮 · 밤 · 젖은 날씨	3 / 3 정지 — 첫 탭 마른 노면, 다음 탭에서 빙판
빙판 · 구간 정보 없음	카메라가 놓쳐 탭 없음 → 30 km/h 충돌
빙판 없음 · 구간 안 · 영하	탭 3번 모두 '이상 없음' — 잘못된 제동 0

CARLA 결함 — 마찰 트리거 안에 다른 차가 있으면 자차가 트리거 밖에서도 그 마찰을 받는다. 정지 차량을 트리거 밖에 두고 모든 실험을 다시 했고, 주행마다 출발 가속으로 마른 출발을 확인했다.

안전 전용 RTOS — 메인 컴퓨터(리눅스)가 멈춰도 2차 방어는 산다

STM32 의 몫은 '더 좋은 제어'가 아니라 고장 격리다. 그 값은 2차 방어가 결과를 바꾸는 장면에서 보인다.



곡선 빙판에서 파이를 죽였다 — 2차 방어가 파이에 있으면 방어 없음과 같고, STM32 에 있으면 고장 없음과 같다

53.1 km/h
2차 방어가 파이 · 파이 정지
= 방어 없음 53.1

39.4 km/h
2차 방어가 안전 전용 RTOS · 파이 정지
≈ 고장 없음 38.9 (μ 0.12 · 60 km/h)

0.12 s
파이 심장박동 끊김 감지 →
축소 모드 (카메라·지도 입력 버림)

전 코어 부하 · 20 ms × 6,000회	평균 지연	최악 지연	마감 놓침
일반 프로그램	2.6 ms	206 ms	211
실시간 우선순위 + 코어 고정	0.04 ms	0.23 ms	0
파이 격리 코어 (RTOS 대체)	0.018 ms	0.06 ms	0



■ 빙판을 확인한 직후 파이 메인 프로그램을 실제로 강제 종료(SIGKILL) — 격리 코어의 2차 방어가 혼자 4.1 m 앞에 세운다

판단이 일어나는 곳

· 데스크톱(CARLA·Moza)이 IMU·거리를 보내고, 파이 격리 코어가 20 ms 마다 탭·정지 거리·제동을 답한다.

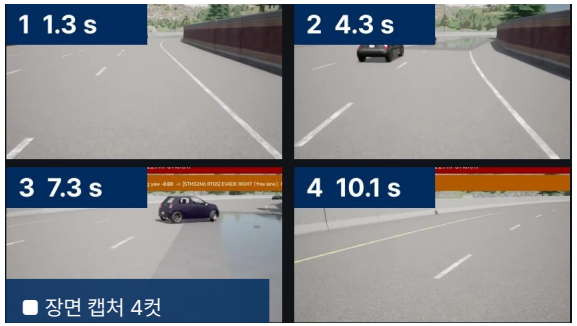
· 캠퍼스 와이파이 왕복 중앙 4.8 ms · 99 % 9.4 ms. 0.2 s 까지의 무응답은 마지막 명령을 유지한다.

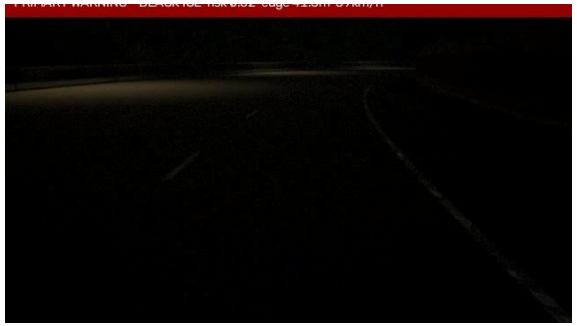
· 격리 코어의 판단 = 보드 C 코드 = 파이썬 참조 (900 샘플 비트 동일).

STM32N6 보드가 없어 같은 C 코드를 라즈베리파이 5 격리 코어(코어 3 · SCHED_FIFO · mlockall)에서 돌려 측정했다. 리눅스 커널 자체가 멈추면 같이 멈춘다 — 실차 설계는 별도 MCU 다.

한계와 실패 사례 — 숨기지 않고 남긴 것









01 경보 거리는 기하 문제다

모델이 요구하는 최소 80 px 과 ROI 해상도 91 px 이 경보 거리를 정한다. 알고리즘이 아니라 해상도를 올리는 것이 먼저다.

02 2차는 예방이 아니다

60 km/h. 1차가 경고했지만 제동이 늦어 진입했고, 2차가 0.26 s 만에 개입했는데도 39.2 km/h 로 앞차와 충돌했다.

03 오경보도 난다

밤. 빙판이 41 m 밖(ROI 7.9~42.2 m 바깥)에 있는데 어두운 노면을 얼음으로 보아 위험도 0.52 로 경보가 났다.

04 질감 도메인 갭

RSCD 실사진 질감을 CARLA 노면에 입히면, 같은 모델이 그 얼음에 반응하지 않는다. 재수집·재학습이 다음 과제다.

그 밖에 남은 것

· Hailo-10H 교차 검증은 Dataflow Compiler 미확보로 열지 못했다. 1차는 STM32N6 Neural-ART 단독 결과다. · D435i 실측 IMU 노이즈는 PVS 공개 데이터로 대체했다. 실물 연결 측정은 남은 과제다.

· 기준선은 제어 상실 후 운전자 입력을 모형화하지 않는다. '아무 보조도 없을 때의 물리적 귀결'로 읽어야 한다. · 젖은 노면 경보율 0.8 %는 더 낮출 여지가 있다. β(반사도 가중치)와 문턱의 재조정이 필요하다.

215 / 6,340
실사진 얼음을 문턱 아래로 놓침 (3.4 %)

368 / 63,018
얼음 아닌 노면에 경보 (0.6 %)

7.9 ~ 42.2 m
카메라 ROI — 이 밖은 보지 못한다

결론 · 재현성 · 향후 계획

무엇을 만들었나

카메라 한 겹에 기대지 않는 이중 안전망을 만들었다. 1차는 빙판에 닿기 전에 세우고, 1차가 놓치면 2차가 노면 마찰을 직접 재서 제동 시점을 노면에 맞춘다(정지 2/9 → 8/9). 2차 판단은 메인 컴퓨터와 분리된 안전 전용 RTOS에서 돌아 리눅스가 멈춰도 계속된다.

재현 — 모든 숫자는 실측에서 자동 생성된다

```
python3 sw/scripts/analysis/summarize_runs.py  # 집계표 재생성
python3 -m pytest -q sw/tests                  # 단위 테스트 19개
python3 sw/fw/npu_lib/test_slip_core.py        # C 코어 ↔ 파이썬 동치
```

23
근거 문서

41
결과 그림

108
주행 이벤트 JSON · 저장소에 함께

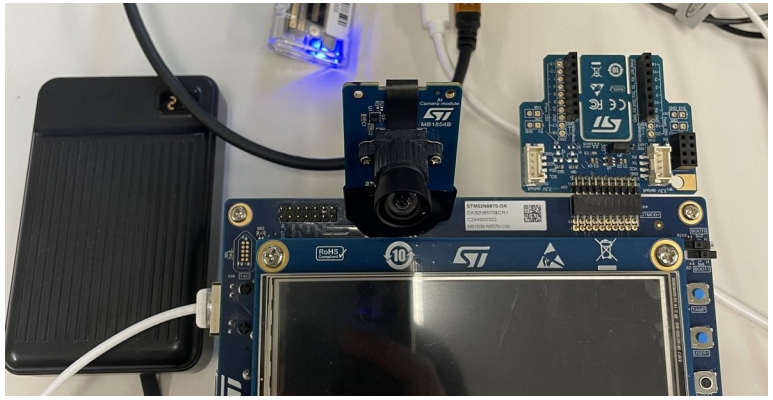
향후 계획

1. 경보 거리 확대 — ROI 해상도를 올려 1차가 보는 거리를 늘린다. 2차 의존도를 낮추는 가장 직접적인 길이다
2. 실사진 질감 재학습 — RSCD 질감을 CARLA 노면에 입혀 재수집하고 도메인 갭을 닫는다
3. Hailo-10H 교차 검증 — Dataflow Compiler 확보 후 동일 모델을 두 NPU 에서 비교
4. 보드 실측 — 마찰 인식 제동 시점·파이 감시를 STM32N6 ThreadX 에서 다시 잰다 (이번 측정은 파이 격리 코어 대체)
5. 운전자가 못 느끼는 마찰 측정 — 앞축 구동·뒷축 제동으로 차 전체 가속을 바꾸지 않고 마찰만 재는 방법

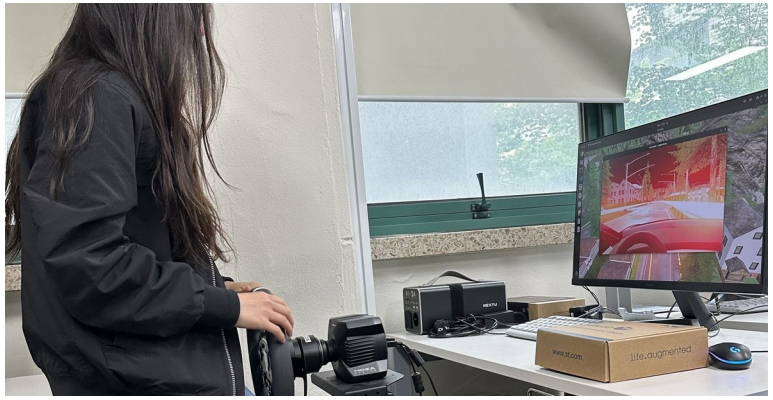
소스 코드

github.com/gkgk0119gmail-arch/2026ESWContest_mobility_ARTS

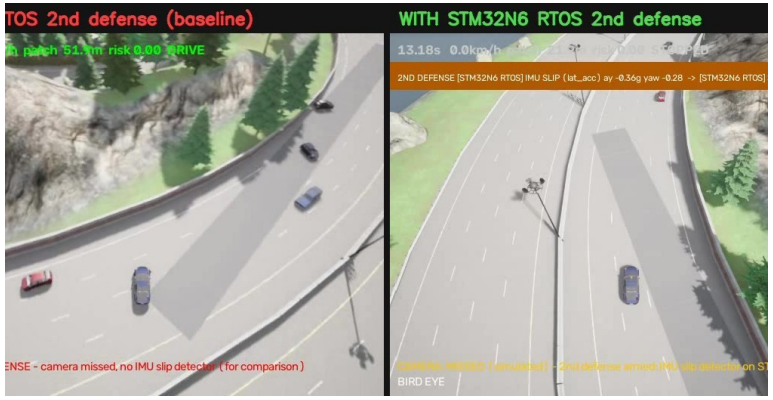
MIT 라이선스 · 영상·그림·이벤트 데이터 포함 · README 에서 바로 재생된다



실물 보드가 실제 펌웨어를 돌린다



사람이 운전해도 보드는 같은 판정을 한다



CARLA 가 센서와 물리를 제공한다 — 같은 조건 비교

팀 ARTS · 이지성 · 남윤상 · 김진찬 — 제24회 임베디드SW경진대회 자동차/모빌리티(현대자동차) 부문